BIP v2: Open-Subgoal Inference with an RL-Trained Proposer

Oct 12, 2026 · @Minh

Summary

BIP v2 infers the whole task structure online, which subgoals exist and how they are ordered, instead of receiving the subgoals at the start. An open-weights LLM proposes candidate structures, the existing particle filter scores each one by its Bayesian evidence, and that score is the reward that trains the LLM by RL to propose structures in proportion to their posterior.

The core ML claim: the evidence from a Bayesian inference engine is a usable, label-free training signal that amortizes open-ended structure inference into an LLM proposer. The trained proposer should reach the true structure faster and with fewer LLM calls than a frozen LLM, in PARTNR simulation and on a real robot.

Contributions:

- Open-subgoal task inference: a hypothesis is a full structure (subgoal specs plus orderings), inferred by trans-dimensional SMC with LLM-proposed birth, death and edit moves.
- RL from Bayesian evidence: a GFlowNet-style objective trains the proposer to sample structures in proportion to prior times evidence, with no structure labels.
- Evaluation in sim and real: structure accuracy, assistance quality and LLM cost, including a proposer trained only on unlabeled recordings.

What changes from the current BIP

The hidden variable grows from the orderings E to the full structure H = (V, E); the filter, likelihoods and scene tracker stay as they are.

Component	Current BIP	BIP v2
Subgoals V	given with the task	latent, part of each hypothesis

Component	Current BIP	BIP v2
Hypothesis	graph G = (V, E)	structure H = (V, E), each subgoal a spec (object, relation, furniture, optional reference)
Prior	Gemini's pairwise orderings, 2.1	frozen base LLM log-probability plus a size penalty on |V|
Pool	64 graphs sampled from the prior	64 structures from the proposer, importance-weighted
S2 revisions	Gemini graphs added without proposal weights	birth, death and edit moves with Metropolis–Hastings acceptance
Proposer	frozen Gemini	open-weights LLM trained by RL against the evidence
Decision	P_exec, Y_v, Δ	adds P(v exists)

The prior is kept separate from the proposer on purpose. If the trained LLM were also the prior, training would move the target it is trained toward. A frozen base model defines the target; the trained copy only proposes.

Candidate subgoal specs come from two sources. Top-down, the proposer reads the instruction and the premapped scene. Bottom-up, any completion the tracker observes that matches no subgoal in a hypothesis spawns one, with its spec read off the scene.

Inference: trans-dimensional SMC over structures

The pool becomes a properly weighted sample of the posterior over structures, so S2's additions no longer bias it. Each structure keeps its own 64-particle filter, unchanged from section 3.3.

Initial pool: draw K = 64 structures from the proposer q, and weight each by prior times evidence over proposal.

```
w_t(H) \propto \frac{p_0(H \mid \ell, s_0)\, \hat Z_t(H)}{q_\theta(H \mid \ell, s_0)}
```

Rejuvenation on surprise: the proposer, conditioned on the leading structures and the event history, suggests a move to H′. The move is replayed through the history, as S2 does now, and accepted with the Metropolis–Hastings ratio.

$$\alpha = \min\left(1,\ \frac{p_0(H')\, \hat Z_t(H')\, q_\theta(H \mid H')}{p_0(H)\, \hat Z_t(H)\, q_\theta(H' \mid H)}\right)$$

Move types:

- Birth: add a subgoal, for example when the person picks up an object no leading structure mentions.
- Death: drop a subgoal that has drawn no evidence after its predecessors are done.
- Edit: change a subgoal's furniture or reference object, or add or remove an ordering.
- Bottom-up birth: an observed completion that matches no subgoal adds one deterministically.

The surprise trigger stays a log Bayes factor, but the reference becomes a null model in which every observable action is equally likely. The old reference (one graph with no orderings) assumes the subgoals are known.

Open question: reverse-move probabilities q(H | H′) need the proposer to score moves it did not generate; scoring them by the LLM's log-probability of the reverse edit is the default.

RL-trained proposer

The proposer is trained to sample structures in proportion to prior times evidence, using recorded episodes and no structure labels. That is what lets it train on real recordings, where labels do not exist.

Reward for a structure H on an episode prefix o_1..t, computed by replaying the prefix through the S1 filter:

$$\log R(H) = \log p_0(H \mid \ell, s_0) + \log \hat Z_t(H)$$

Objective: trajectory balance, a GFlowNet loss, rather than reward maximization. Reward maximization collapses onto the single best structure; the filter needs a spread of plausible ones.

$$\mathcal{L}(\theta) = \big(\log Z_\phi + \log q_\theta(H) - \log R(H)\big)^2$$

Training loop:

1. Sample a recorded episode and a cut point t, so the proposer learns the posterior at every stage of a task, not only at the end.

2. Sample structures from q (on-policy) and from a replay buffer of high-reward ones (off-policy; trajectory balance allows both).
3. Replay the prefix through S1 for each structure to get log Ẑ_t; this is CPU work, cheap next to LLM sampling.
4. Take a gradient step on the loss; one model serves both the initial proposal and the revision moves, with the history in its context.

Design details:

- Canonical output order: subgoals sorted by spec, edges sorted, so each structure has one token sequence and q(H) is exact.
- Reward noise: the filter gives an unbiased Ẑ, so log Ẑ is biased low. Use more particles during training than at run time, and average over filter seeds.
- Output format: JSON constrained to the scene's objects and furniture, so every proposal is a valid spec.

Decision rule

The only change is a new risk: helping with a subgoal the person never wanted. Subgoals are matched across structures by spec (object and furniture), and each gets an existence probability.

```
P_{\text{exist}}(v) = \sum_H w_t(H)\, [\, v \in V(H) \,]
```

```
V(\text{execute } v) = P_{\text{exist}}\Big(P_{\text{exec}}\big[(1-
Y_v)\,\Delta - \beta\, Y_v\big] - \beta_{\text{prec}}\,(1-
P_{\text{exec}})\Big) - \beta_{\text{wrong}}\,\big(1-P_{\text{exist}}\big)
```

β_wrong, the cost of moving an object that should have stayed, is set relative to d like the other costs. A subgoal with P_exist below 0.5 is never executed, alongside the rules of section 6.5. The rule stays hand-designed, so any gain in assistance comes from better inference, not a better policy.

Experiments

The headline comparison is RL from evidence against supervised fine-tuning on true structures: if the label-free proposer matches SFT in sim and beats it on real data, the claim holds.

Settings:

- Sim (PARTNR): train the proposer on replays from training scenes; test on held-out scenes and held-out task types. Run each method with subgoals given and with subgoals open.
- Real: robot plus smart glasses, participants doing household tasks. Two proposers: trained in sim only, and the same then fine-tuned on unlabeled real recordings, which only the RL proposer can use.

Method	Proposer	Needs structure labels
BIP v2 (ours)	open LLM, RL from evidence	no
SFT proposer	open LLM, fine-tuned on true structures	yes
Frozen base	open LLM, untrained	no
Frozen Gemini	current BIP prior, extended to propose subgoals	no
Subgoals given	current BIP	yes (an upper reference)
End-to-end VLM	VLM watches the video and decides directly	no
No help	person works alone	no

Metric group	Measures
Structure	subgoal-set F1, ordering F1, log posterior of the true structure, time until it leads the pool
Assistance	team completion time, person time saved, conflicts, ordering violations, wrong-subgoal actions
Cost	LLM calls, tokens and latency per episode

Ablations: trajectory balance versus reward maximization (does the spread matter?), no birth or death moves, no bottom-up births, no gaze, proposer size, and particles per structure.

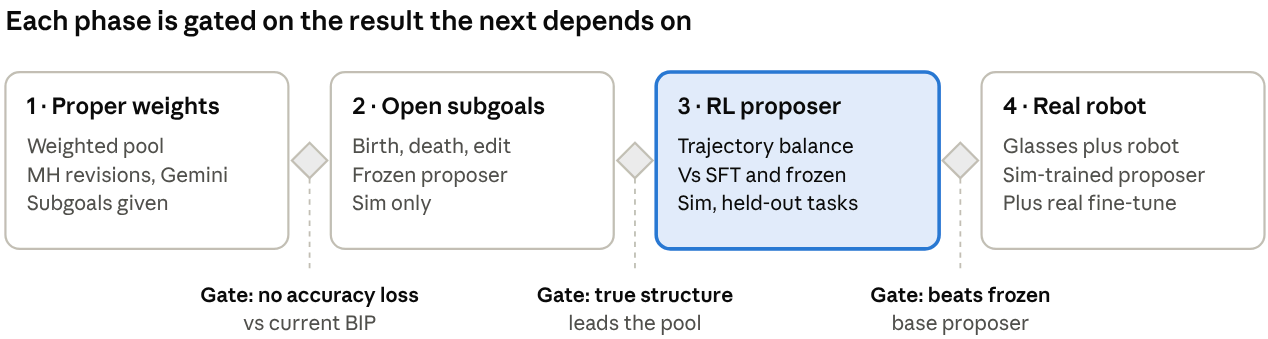
Open question: the number of real-world participants and tasks, which sets how much of the real-robot claim is statistical.

Risks and milestones

The biggest risk is a reward too noisy to train on; it is tested in phase 3 before any real-robot work depends on it.

Risk	Mitigation
Noisy log Ẑ reward stalls training	more particles at training time, seed averaging, replay buffer of high-reward structures
Size penalty mis-set: the posterior invents subgoals or drops real ones	tune the penalty on held-out sim tasks; report sensitivity
Subgoals never acted on stay invisible until done	bottom-up births; P_exist keeps the robot off them
Reverse-move scores q(H | H′) are poor, so MH mixes slowly	score reverse edits by log-probability; fall back to importance-weighted additions
Real action recognition is far noisier than sim	fine-tune on unlabeled real recordings; report recognition error alongside results
Compute for RL on an open LLM	start with a small model and short JSON outputs; reward computation is CPU-bound

Milestones run in four phases, each gated on a result that the next one depends on.



milestone roadmap · 4 phases, 3 gates

Phase 1 fixes the pool's weighting with subgoals still given, so any later gain is attributable to open subgoals or to RL. Phase 3 is the paper's core result; phase 4 only starts once it holds in sim.